

On the Criteria To Be Used in Decomposing Systems into Modules

D.L. Parnas

Parnas começa o artigo observando que todo mundo fala sobre os *benefícios* de dividir um sistema em módulos, mas quase ninguém fala sobre *como* decidir onde cortar. É exatamente essa decisão que ele quer discutir.

A ideia é que separar um sistema em módulos traz três vantagens: times diferentes podem trabalhar em paralelo sem precisar se comunicar muito, mudanças em uma parte não deveriam quebrar as outras, e qualquer pessoa consegue entender o sistema estudando um módulo de cada vez.

Para mostrar que o critério de corte importa muito, Parnas pega um sistema simples de índice de palavras (o KWIC) e propõe duas formas de dividi-lo em módulos.

Na primeira decomposição, cada etapa do processamento vira um módulo: leitura, deslocamento circular, ordenação alfabética e saída. É exatamente o que a maioria dos programadores faria desenhando um fluxograma. O problema é que os módulos compartilham representações internas de dados, como o formato de armazenamento das linhas em memória. Se você mudar essa representação, precisa alterar todos os módulos ao mesmo tempo.

Na segunda decomposição, o critério é diferente. Cada módulo existe para esconder uma decisão de design das demais partes do sistema. O módulo de armazenamento de linhas, por exemplo, expõe apenas funções de acesso como $\text{CHAR}(r,w,c)$ e esconde completamente como os dados estão guardados internamente. Ninguém de fora sabe se os caracteres estão empacotados quatro por palavra ou não. O mesmo vale para o módulo de deslocamento circular: ele expõe uma interface simples e esconde se os deslocamentos são pré-computados ou calculados na hora.

Parnas lista decisões que provavelmente vão mudar ao longo da vida do sistema, como o formato de entrada, a decisão de guardar tudo em memória, o esquema de empacotamento de caracteres. Na primeira decomposição, qualquer uma dessas mudanças afeta todos os módulos. Na segunda, cada mudança fica contida dentro do único módulo que "sabe" sobre aquela decisão.

Além disso, o desenvolvimento independente fica mais fácil porque as interfaces entre módulos na segunda versão são abstratas (nomes de funções e parâmetros), não estruturas de dados complexas que todo time precisa concordar antes de começar a trabalhar.

O nome que Parnas dá ao critério é *information hiding*, esconder informação. Um módulo bem desenhado não é uma etapa de processamento, mas sim o guardião de uma decisão de design. Sua interface deve revelar o mínimo possível sobre como ele funciona por dentro.

Ele também observa que módulos com esse critério tendem a formar uma hierarquia natural de dependências: módulos de nível baixo funcionam sem os de cima, e é possível reaproveitar as partes inferiores em outros sistemas completamente diferentes.

A segunda abordagem, se implementada ingenuamente com chamadas de função normais, pode ser mais lenta por causa da frequência dessas chamadas. Parnas reconhece isso e sugere que a solução é tratar módulos como unidades conceituais de organização, não necessariamente como unidades de compilação, usando assemblers ou ferramentas que ensinam o código da forma mais eficiente para cada caso.

Decompor um sistema seguindo o fluxograma de execução quase sempre é errado. O ponto de partida correto é listar as decisões de design que são difíceis ou propensas a mudar, e então criar um módulo para esconder cada uma delas. Feito assim, mudanças ficam contidas, o desenvolvimento em paralelo fica mais simples, e o sistema como um todo é mais fácil de entender.